

AULA 03: LINGUAGENS REGULARES, AUTÔMATOS FINITOS, EXPRESSÕES E GRAMÁTICAS

A Tríade da Computação Regular

Prof. Marcos Ronaldo Melo Cavaleiro | UNIJUÍ – Ciência da Computação

Como este material foi construído



01. Uso de I.A.

Gemini (Google NotebookLM)

Inteligência Artificial aplicada exclusivamente na etapa de planejamento inicial para estruturar rascunhos conceituais a partir do material de aula.

A ferramenta atua como assistente para estruturação de base bruta, não gerando o produto final.



02. Curadoria Docente

Responsabilidade do Professor

Todo o conteúdo gerado por IA passa por rigorosa revisão humana e pedagógica do professor antes de ser apresentado.

Lembrete: O mesmo rigor ético e analítico de revisão é esperado de vocês no uso de IA para os EIs.

Slide 1: Planejamento da Aula & Paradigma de Estados Finitos

Objetivos & Cronograma

- Compreender o modelo de Sistemas de Estados Finitos.
- Formalizar AFD, AFN e AFN com transições vazias (AFN- ϵ).
- Aplicar o Algoritmo da Construção de Subconjuntos.
- Especificar linguagens via Expressões Regulares (ER) e Gramáticas Regulares (GR).
- Bloco 1 (20m): Sistemas & AFD | Quiz 1 (10m)
- Bloco 2 (20m): AFN, AFN- ϵ & Subconjuntos | Quiz 2 (10m)
- Bloco 3 (15m): ER & GR | Quiz 3 (10m)
- Bloco 4 (10m): Lab / Peer Instruction | Fechamento (5m)

O Paradigma de Estados Finitos

- Estado: Resume toda a informação do passado necessária para decidir o comportamento futuro.
- Exemplo Elevador: Memoriza apenas o andar atual e a direção, sem histórico de viagens.
- Exemplo Analisador Léxico: Memoriza o prefixo lido para decidir o token (palavra reservada, ID, número).
- Limitação Estrutural: Memória fixa e pré-definida. Não há fita ou pilha para contagem arbitrária.

Slide 3: Autômato Finito Determinístico (AFD)

Definição Formal

Um AFD é uma quintupla matemática $M = (\Sigma, Q, \delta, q_0, F)$:

- Σ : Alfabeto finito e não vazio de símbolos de entrada.
- Q : Conjunto finito e não vazio de estados.
- $\delta : Q \times \Sigma \rightarrow Q$: Função total/parcial de transição.
- $q_0 \in Q$: Estado inicial único.
- $F \subseteq Q$: Conjunto de estados finais (aceitação).

Função de Transição Estendida (δ^*)

A função δ^* processa cadeias completas $w \in \Sigma^*$:

1. Base de Indução:

$$\delta^*(q, \epsilon) = q$$

2. Passo Indutivo:

$$\delta^*(q, aw) = \delta^*(\delta(q, a), w), \text{ para } a \in \Sigma, w \in \Sigma^*$$

- Aceitação: Uma palavra w é aceita se $\delta^*(q_0, w) \in F$.
- Rejeição: Caso contrário, w é rejeitada.

Slide 4: Exemplo Prático – Construção de um AFD

Problema: Reconhecer subpalavra 'aa' ou 'bb' sobre $\Sigma = \{a, b\}$

$L = \{ w \in \{a, b\}^* \mid w \text{ possui 'aa' ou 'bb' como subpalavra} \}$

Tabela de Transição δ :

- Estado Inicial $\rightarrow q_0$: [a $\rightarrow$ q1, b $\rightarrow$ q2] (Nenhum símbolo Relevante)
- Estado q1: [a $\rightarrow$ qf, b $\rightarrow$ q2] (Último símbolo foi 'a')
- Estado q2: [a $\rightarrow$ q1, b $\rightarrow$ qf] (Último símbolo foi 'b')
- Estado Final *qf: [a $\rightarrow$ qf, b $\rightarrow$ qf] (Subpalavra encontrada - Estado Poço)

Rastreio da palavra 'abba':

$\delta^*(q_0, abba) = \delta^*(q_1, bba) = \delta^*(q_2, ba) = \delta^*(qf, a) = qf \in F$ (ACEITA)

Slide 4: Exemplo Prático – Construção de um AFD

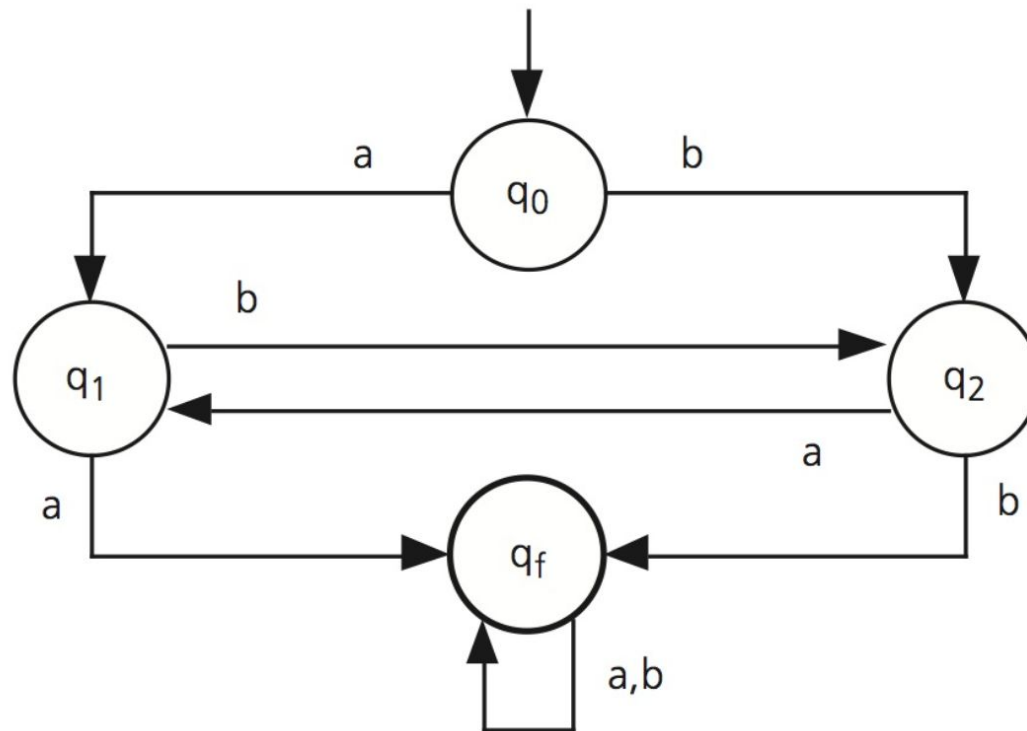
Tabela de Transição:

δ_1	a	b
q ₀	q ₁	q ₂
q ₁	q _f	q ₂
q ₂	q ₁	q _f
q _f	q _f	q _f

Fonte: <https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/70>

Slide 4: Exemplo Prático – Construção de um AFD

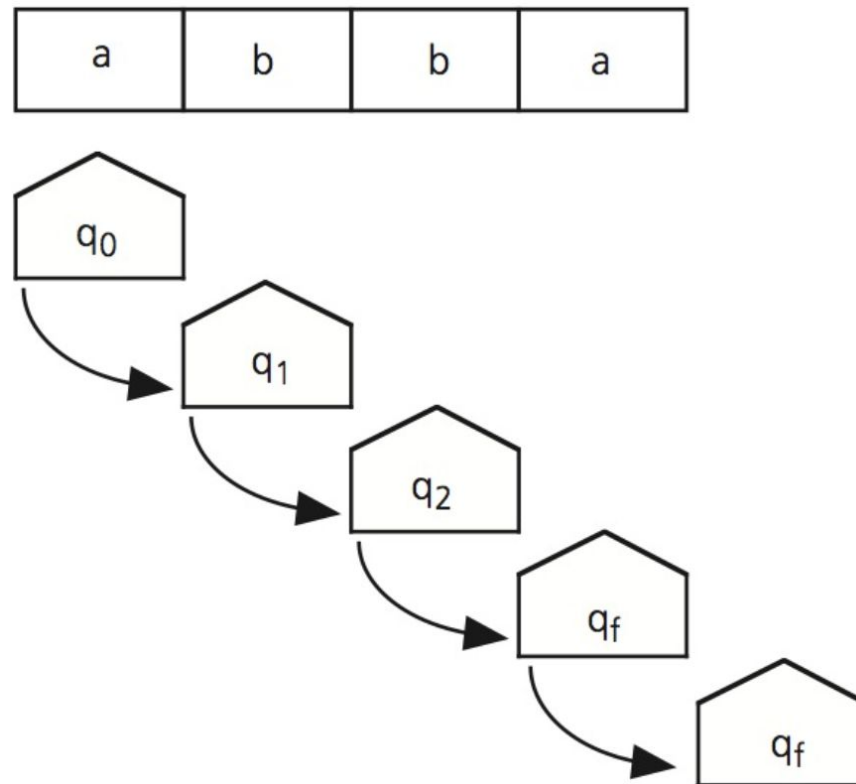
Diagrama (AFD): sequência de dois símbolos iguais.



Fonte: <https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/71>

Slide 4: Exemplo Prático – Construção de um AFD

Diagrama (AFD): Teste da palavra “abb”.



Fonte: <https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/71>

Slide 5: Checkpoint Rápido 1 (Quiz Interativo)

Questões de Verificação

1. Computação de Função Estendida:

Dado o AFD do Slide 4, qual é o resultado do rastreo de $\delta^*(q_0, abba)$?

- (A) q_1
- (B) q_2
- (C) q_f (Aceita)
- (D) Indefinido

2. Cadeia Vazia (ϵ):

Sob que condição a palavra vazia ϵ pertence à linguagem $L(M)$?

- (A) Somente se $|Q| = 1$.
- (B) Se e somente se $q_0 \in F$.
- (C) Se $\delta(q_0, a) = q_0$ para todo a .
- (D) ϵ nunca pode ser aceita por AFD.

Respostas?

Slide 6: Autômato Finito Não-Determinístico (AFN)

Conceito & Definição Formal

- No Não-Determinismo, para um estado e símbolo, o autômato pode transitar para ZERO, UM ou MÚLTIPLOS estados simultaneamente.
- $M = (\Sigma, Q, \delta, q_0, F)$
- Função de Transição: $\delta : Q \times \Sigma \rightarrow 2^Q$
- A imagem de δ é o Conjunto das Partes (Power Set) de Q .
- Aceitação: Uma palavra é aceita se AO MENOS UM ramo de execução terminar em um estado de aceitação.

δ_5	a	b
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_f\}$	$\emptyset$
q_2	$\emptyset$	$\{q_f\}$
q_f	$\{q_f\}$	$\{q_f\}$

Exemplo: Linguagem com Sufixo 'aaa'

$$L = \{ w \in \{a, b\}^* \mid w \text{ termina em 'aaa'} \}$$

Transições:

- $\delta(q_0, a) = \{q_0, q_1\}$
- $\delta(q_0, b) = \{q_0\}$
- $\delta(q_1, a) = \{q_2\}$
- $\delta(q_2, a) = \{q_f\}$
- $\delta(q_f, a) = \emptyset, \delta(q_f, b) = \emptyset$

Visão Intuitiva: Permanecemos em q_0 e 'palpitamos' o início do sufixo 'aaa'.

Slide 6: Autômato Finito Não-Determinístico (AFN)

Conceito & Definição Formal

- No Não-Determinismo, para um estado e símbolo, o autômato pode transitar para ZERO, UM ou MÚLTIPLOS estados simultaneamente.
- $M = (\Sigma, Q, \delta, q_0, F)$
- Função de Transição: $\delta : Q \times \Sigma \rightarrow 2^Q$
- A imagem de δ é o Conjunto das Partes (Power Set) de Q .
- Aceitação: Uma palavra é aceita se AO MENOS UM ramo de execução terminar em um estado de aceitação.

δ_5	a	b
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_f\}$	$\emptyset$
q_2	$\emptyset$	$\{q_f\}$
q_f	$\{q_f\}$	$\{q_f\}$

Exemplo: Linguagem com Sufixo 'aaa'

$$L = \{ w \in \{a, b\}^* \mid w \text{ termina em 'aaa'} \}$$

Transições:

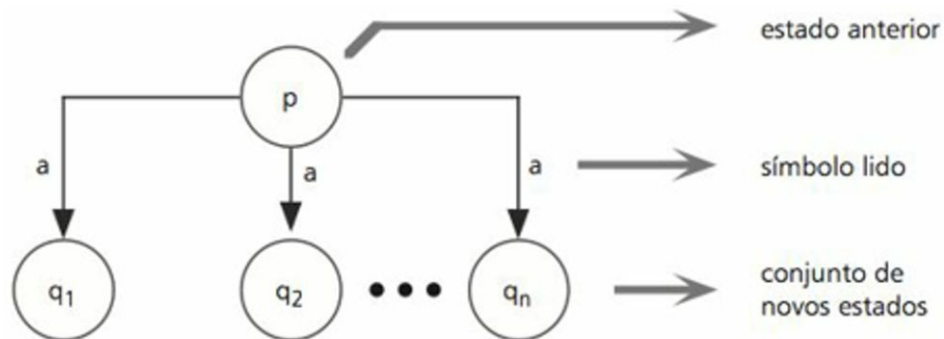
- $\delta(q_0, a) = \{q_0, q_1\}$
- $\delta(q_0, b) = \{q_0\}$
- $\delta(q_1, a) = \{q_2\}$
- $\delta(q_2, a) = \{q_f\}$
- $\delta(q_f, a) = \emptyset, \delta(q_f, b) = \emptyset$

Visão Intuitiva: Permanecemos em q_0 e 'palpitamos' o início do sufixo 'aaa'.

Slide 6: Autômato Finito Não-Determinístico (AFN)

Conceito & Definição Formal

- No Não-Determinismo, para um estado e símbolo, o autômato pode transitar para ZERO, UM ou MÚLTIPLOS estados simultaneamente.
- $M = (\Sigma, Q, \delta, q_0, F)$
- Função de Transição: $\delta : Q \times \Sigma \rightarrow 2^Q$
- A imagem de δ é o Conjunto das Partes (Power Set) de Q .
- Aceitação: Uma palavra é aceita se AO MENOS UM ramo de execução terminar em um estado de aceitação.



Exemplo:

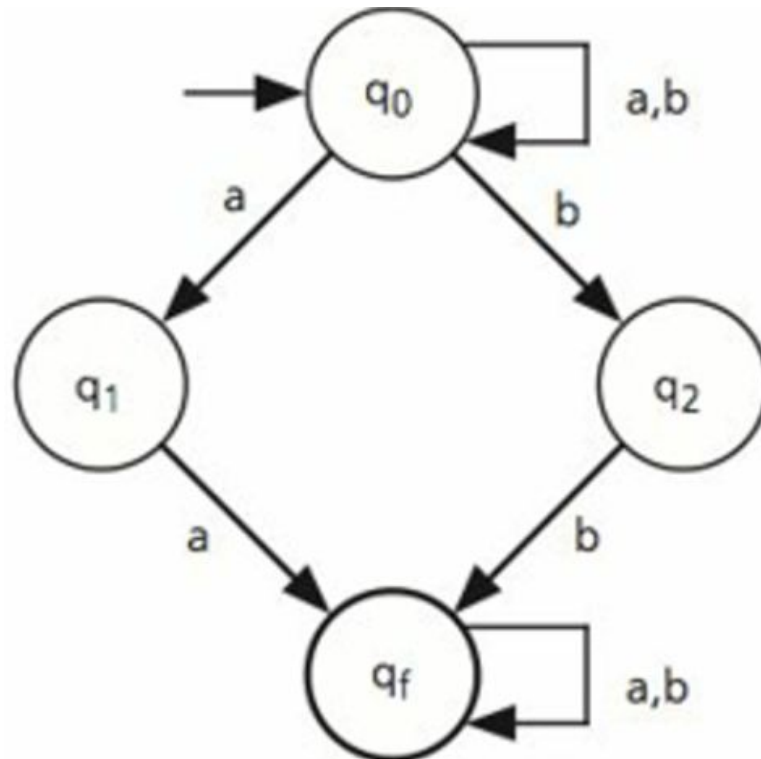
δ_s	a	b
q_0	$\{q_0, q_1\}$	$\{q_0, q_2\}$
q_1	$\{q_f\}$	$\emptyset$
q_2	$\emptyset$	$\{q_f\}$
q_f	$\{q_f\}$	$\{q_f\}$

Fonte:

<https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/71>

Slide 6: Autômato Finito Não-Determinístico (AFN)

Diagrama (AFN): Sequencia de dois símbolos iguais

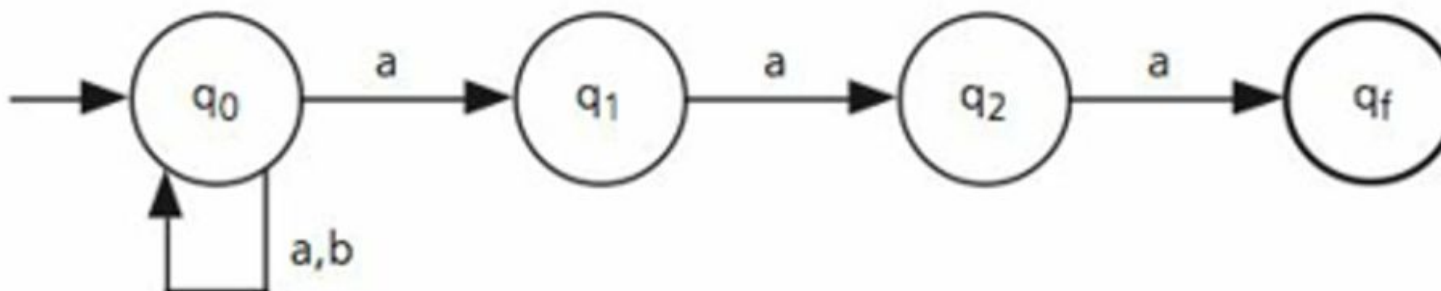


Fonte:

<https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/81>

Slide 6: Autômato Finito Não-Determinístico (AFN)

Diagrama de AFD a partir do AFN



Fonte: <https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/85>

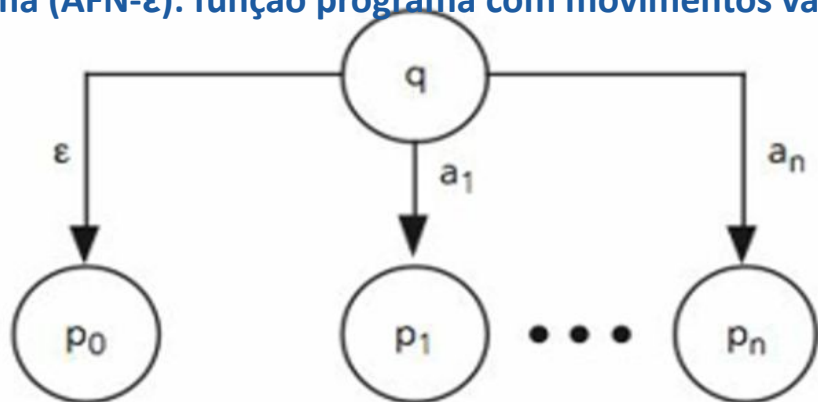
Slide 7: AFN com Movimentos Vazios (AFN- ϵ) & Fecho- ϵ

Movimentos Vazios (AFN- ϵ)

- Permite transições rotuladas com ϵ (mudar de estado sem consumir nenhum símbolo da entrada).
- Fecho Vazio $\delta_\epsilon(q)$: Conjunto de todos os estados alcançáveis a partir de q usando ZERO ou MAIS transições ϵ .
- Para um conjunto $P \subseteq Q$:

$$\delta_\epsilon(P) = \bigcup_{q \in P} \delta_\epsilon(q)$$
- A propriedade reflexiva garante que $q \in \delta_\epsilon(q)$.

Diagrama (AFN- ϵ): função programa com movimentos vazio



Fonte: <https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/86>

Teorema da Equivalência Fundamental

Classe AFD $\equiv$ Classe AFN $\equiv$ Classe AFN- ϵ

- Todos os três modelos possuem exatamente o mesmo poder computacional e reconhecem a mesma classe de linguagens (Linguagens Regulares).

Teorema (Construção de Subconjuntos):

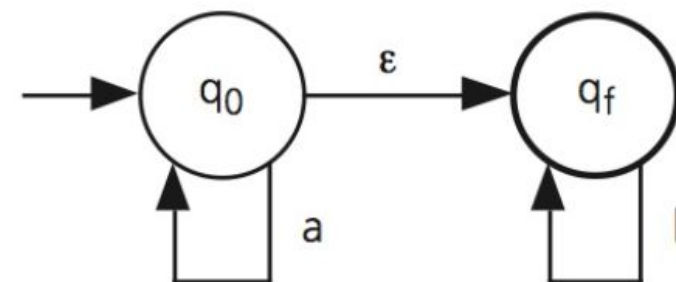
- Todo AFN/AFN- ϵ com n estados pode ser convertido em um AFD equivalente com até 2^n estados.

Slide 7: AFN com Movimentos Vazios (AFN- ϵ) & Fecho- ϵ

Função programa (AFN ϵ): qualquer símbolo a antecede qualquer símbolo b.

δ_7	a	b	ϵ
q_0	$\{q_0\}$	$\emptyset$	$\{q_f\}$
q_f	$\emptyset$	$\{q_f\}$	$\emptyset$

Diagrama (AFN ϵ): qualquer símbolo a antecede qualquer símbolo b.



Fonte: <https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/87>

Slide 7: AFN com Movimentos Vazios (AFN- ϵ) & Fecho- ϵ

Função programa (AFN ϵ): qualquer símbolo a antecede qualquer símbolo b.

δ_7	a	b	ϵ
q_0	$\{q_0\}$	$\emptyset$	$\{q_f\}$
q_f	$\emptyset$	$\{q_f\}$	$\emptyset$

Diagrama (AFN ϵ): qualquer símbolo a antecede qualquer símbolo b.

Fonte: <https://integrada.minhabiblioteca.com.br/reader/books/9788577807994/pageid/87>

Slide 8: Algoritmo da Construção de Subconjuntos (AFN $\rightarrow$ AFD)

Passo a Passo Algorítmico

Entrada: AFN $M = (\Sigma, Q, \delta, q_0, F)$ | Saída: AFD $M_D = (\Sigma, Q_D, \delta_D, q_{0_D}, F_D)$

1. Estado Inicial do AFD:

$$q_{0_D} = \delta^*(\{q_0\})$$

2. Novas Transições: Para cada estado $S \in Q_D$ e cada símbolo $a \in \Sigma$:

$$\delta_D(S, a) = \delta^*(\bigcup_{q \in S} \delta(q, a))$$

3. Repetir o processo até que nenhum novo conjunto/estado seja gerado em Q_D .

4. Estados Finais:

$$F_D = \{S \in Q_D \mid S \cap F \neq \emptyset\}$$

(qualquer subconjunto contendo ao menos um estado final original).

Aplicação no Exemplo do Sufixo 'aaa':

• $S_0 = \langle q_0 \rangle \xrightarrow{a} S_1 = \langle q_0, q_1 \rangle \xrightarrow{a} S_2 = \langle q_0, q_1, q_2 \rangle \xrightarrow{a} S_3 = \langle q_0, q_1, q_2, q_f \rangle$ (Final!)

Slide 9-a: Checkpoint Rápido 2 (Quiz Interativo)

Questões de Verificação

1. Cálculo de Fecho- ϵ :

Dado um AFN- ϵ com $\delta(q_0, \epsilon) = \{q_1\}$ e $\delta(q_1, \epsilon) = \{q_2\}$, qual é o conjunto $\delta\epsilon(q_0)$?

(A) $\{q_0\}$ | (B) $\{q_1, q_2\}$ | (C) $\{q_0, q_1, q_2\}$ | (D) $\emptyset$

2. Complexidade do Determinismo:

Se um AFN possui $n = 5$ estados, qual o número MÁXIMO de estados do AFD equivalente?

(A) 5 | (B) 10 | (C) 25 | (D) 32 (2^5)

Respostas:

Slide 9-b: Checkpoint Rápido 2 (Quiz Interativo)

Questões de Verificação

1. Cálculo de Fecho- ϵ :

Dado um AFN- ϵ com $\delta(q_0, \epsilon) = \{q_1\}$ e $\delta(q_1, \epsilon) = \{q_2\}$, qual é o conjunto $\delta\epsilon(q_0)$?

- (A) $\{q_0\}$
- (B) $\{q_1, q_2\}$
- (C) $\{q_0, q_1, q_2\}$
- (D) $\emptyset$

2. Complexidade do Determinismo:

Se um AFN possui $n = 5$ estados, qual o número MÁXIMO de estados do AFD equivalente?

- (A) 5
- (B) 10
- (C) 25
- (D) 32 (2^5)

Respostas:

Slide 10: Expressões Regulares (ER) – Formalismo Denotacional

Definição Indutiva

Base de Indução:

- $\emptyset$: Denota a linguagem vazia $\{\}$.
- ϵ : Denota a linguagem $\{\epsilon\}$.
- x ($x \in \Sigma$) : Denota a linguagem $\{x\}$.

Passo Indutivo (se r, s são ERs para R, S):

- União ($r + s$ ou $r \mid s$): Denota $R \cup S$.
- Concatenação ($r \cdot s$ ou rs): Denota $R \cdot S$.
- Fecho de Kleene (r^*): Denota $R^* = R^0 \cup R^1 \cup R^2 \dots$

Precedência de Operadores

Ordem de Precedência (Maior $\rightarrow$ Menor):

1. Fecho de Kleene ($*$)
2. Concatenação ($\cdot$)
3. União ($+$)

Exemplo de Avaliação:

- $a + bc^*$ é interpretado como $a + (b \cdot (c^*))$
- Parênteses podem ser usados para alterar a precedência natural: $(a + b)c^*$

Slide 11: Expressões Regulares e Linguagem geradas

Expressão Regular	Linguagem Gerada
aa	somente a palavra aa
ba*	todas as palavras que iniciam por b , seguido por zero ou mais a
(a + b)*	todas as palavras sobre o alfabeto { a, b }
(a + b)*aa(a + b)*	todas as palavras contendo aa como subpalavra
a*ba*ba*	todas as palavras contendo exatamente dois b
(a + b)*(aa + bb)	todas as palavras que terminam com aa ou bb
(a + ε)(b + ba)*	todas as palavras que não possuem dois a consecutivos

<> Padrões Práticos de ER

Identificadores (C / Java / Python)

letter (letter + digit)*

Regra padrão para validação e reconhecimento de nomes de variáveis e identificadores.

Números Reais (Float)

digit+ '.' digit+

Análise e reconhecimento de valores com ponto flutuante na sintaxe das linguagens.

Slide 12: Gramáticas Regulares (GR) – Formalismo Axiomático

Gramática Regular

Tipo 3 de Chomsky

Uma Gramática $G = (V, T, P, S)$ é Regular se todas as suas regras de produção forem estritamente de um dos tipos:

Linear à Direita (GLD)

$$A \rightarrow wB \text{ ou } A \rightarrow w$$

onde $A, B \in V, w \in T^*$

Linear à Esquerda (GLE)

$$A \rightarrow Bw \text{ ou } A \rightarrow w$$

onde $A, B \in V, w \in T^*$

Regra Crítica de Integridade

ATENÇÃO PEDAGÓGICA

- Uma gramática regular deve ser **EXCLUSIVAMENTE** linear à direita ou **EXCLUSIVAMENTE** linear à esquerda.
- Misturar produções GLD e GLE na mesma gramática pode extrapolar as linguagens regulares e gerar linguagens livres de contexto (ex: $anbn$).

Slide 13: Conversão Direta: Gramática Regular $\leftrightarrow$ AFN

Algoritmo de Mapeamento Direto (GLUD $\rightarrow$ AFN- ϵ)

Dada a Gramática $G = (V, T, P, S)$, construímos $M = (\Sigma, Q, \delta, q_0, F)$ onde $\Sigma = T$, $Q = V \cup \{q_f\}$, $q_0 = S$, $F = \{q_f\}$:

Mapeamento de Produções:

- Produção: $A \rightarrow \epsilon$ $\longrightarrow$ Transição: $\delta(A, \epsilon) = q_f$
- Produção: $A \rightarrow a$ $\longrightarrow$ Transição: $\delta(A, a) = q_f$
- Produção: $A \rightarrow B$ $\longrightarrow$ Transição: $\delta(A, \epsilon) = B$
- Produção: $A \rightarrow aB$ $\longrightarrow$ Transição: $\delta(A, a) = B$

Esse mapeamento simples e isomórfico garante a equivalência direta entre gramáticas e autômatos.

Slide 14: Checkpoint Rápido 3 (Quiz Interativo)

Questões de Verificação

1. Equivalência de ER e Gramática:

Qual ER gera a mesma linguagem da gramática $S \rightarrow aA, A \rightarrow bA \mid \epsilon$?

- (A) $a + b^*$
- (B) ab^*
- (C) $(ab)^*$
- (D) a^*b^*

2. Violação do Formalismo Regular:

Por que a gramática $S \rightarrow aA, A \rightarrow Sb, S \rightarrow \epsilon$ NÃO garante ser regular?

- (A) Por ter a regra $S \rightarrow \epsilon$.
- (B) Por ter mais de duas variáveis.
- (C) Por misturar produções GLD ($S \rightarrow aA$) e GLE ($A \rightarrow Sb$).
- (D) Por ter alfabeto pequeno.

Resposta:

Slide 15: Laboratório Prático / Peer Instruction (20 min)

Desafio em Duplas (Método Mazur)

Cenário: Construir os formalismos para a linguagem L sobre $\Sigma = \{0, 1\}$ das cadeias que NÃO contêm '00' consecutivo.

Etapas:

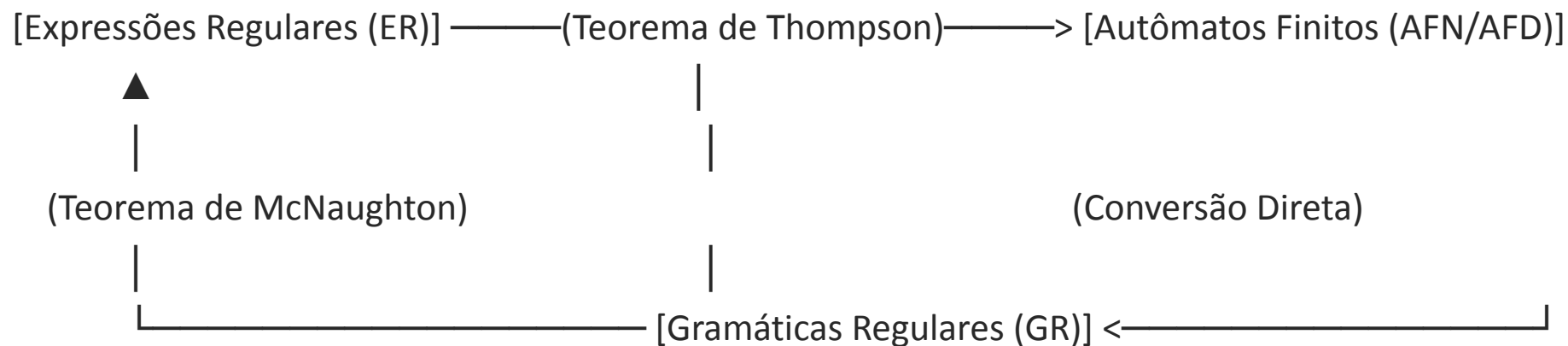
1. Individual (5 min): Tente esboçar o AFD e a ER.
2. Discussão em Duplas (10 min): Compare as ideias.
3. Consenso e Votação (5 min): Apresentação da solução.

Resolução Teórica

- Expressão Regular: $(1 + 01)^*(\epsilon + 0)$
- Estados do AFD:
 - q0 (Inicial): último não foi '0' [0 $\rightarrow$ q1, 1 $\rightarrow$ q0]
 - q1: último foi '0' [0 $\rightarrow$ q_erro, 1 $\rightarrow$ q0]
 - q_erro (Poço): recebeu '00' [0 $\rightarrow$ q_erro, 1 $\rightarrow$ q_erro]
- $F = \{q0, q1\}$
- Gramática Regular GLD:
 - $S \rightarrow 1S \mid 0A \mid \epsilon$
 - $A \rightarrow 1S \mid \epsilon$

Slide 16: Síntese da Aula – A Equivalência Triangular

A Tríade da Computação Regular



- Formalismo Operacional: Autômato Finito (Como reconhecer)
- Formalismo Denotacional: Expressão Regular (O que reconhecer)
- Formalismo Axiomático: Gramática Regular (Como gerar)
- Todos os três modelos possuem rigorosamente o MESMO PODER EXPRESSIVO.

Slide 17: Desafios para Casa

Atividades de Consolidação

Desafio 1 (Subconjuntos): Dado o AFN $M = (\{a, b\}, \{q_0, q_1, q_2\}, \delta, q_0, \{q_2\})$ com $\delta(q_0, a) = \{q_0, q_1\}$, $\delta(q_0, b) = \{q_0\}$, $\delta(q_1, b) = \{q_2\}$, converta-o para AFD.

Desafio 2 (Análise Léxica): Escreva uma ER e desenhe um AFD que reconheça comentários de linha única em C++/Java (iniciando com `//` e terminando em `\n`).

Desafio 3 (Reflexão Teórica): Toda linguagem finita é necessariamente regular? Justifique formalmente utilizando união finita de palavras.

Slide 18: Referências Bibliográficas e Leituras Recomendadas

Fontes Principais & Leituras

- MENEZES, Paulo Blauth. Linguagens Formais e Autômatos. 6ª edição. Porto Alegre: Bookman, 2011. (Capítulo 3: Linguagens Regulares).
- AHO, Alfred V. et al. Compiladores: Princípios, Técnicas e Ferramentas. 2ª edição. São Paulo: Pearson Addison-Wesley, 2007. (Capítulo 3: Análise Léxica).
- HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introdução à Teoria de Autômatos, Linguagens e Computação. Rio de Janeiro: Campus, 2002.
- Plano de Ensino da disciplina de Linguagens Formais e Compiladores – UNIJUÍ.